

```

public class PremiumCalculator {
    public class PremiumInputWrapper {
        @InvocableVariable(required=true) public Id policyId;
    }

    @InvocableMethod(
        label='Calculate Policy Premium'
        description='Calculates premium based on policy fields (VIN, Model Year, State).'
    )
    public static List<Decimal> calculatePremium(List<PremiumInputWrapper> inputs) {

        List<Decimal> premiumResults = new List<Decimal>();
        Set<Id> policyIds = new Set<Id>();

        for (PremiumInputWrapper input : inputs) {
            if (input.policyId != null) {
                policyIds.add(input.policyId);
            }
        }

        List<Policy__c> policiesToRate = [
            SELECT Id, VIN__c, Model_Year__c, Policy_State__c
            FROM Policy__c
            WHERE Id IN :policyIds
        ];

        for (Policy__c policy : policiesToRate) {
            Decimal basePremium = 1000.00;

            if (policy.Policy_State__c == 'CA') {
                basePremium *= 1.15;
            } else if (policy.Policy_State__c == 'TX') {
                basePremium *= 1.05;
            }

            if (policy.Model_Year__c != null &&
                Integer.valueOf(policy.Model_Year__c) < 2018) {
                basePremium += 100;
            }

            premiumResults.add(basePremium);
        }
        return premiumResults;
    }
}

```

```

public class ClaimsAdjusterController {
    public class ClaimWrapper {
        @AuraEnabled public Id claimId;
        @AuraEnabled public String claimNumber;
        @AuraEnabled public Decimal claimAmount;
        @AuraEnabled public String status;
        @AuraEnabled public String policyType;
        @AuraEnabled public String policyHolderName;
        @AuraEnabled public Integer daysOpen;

        public ClaimWrapper(Claim__c c, Policy__c p, Contact customerRec) {
            this.claimId = c.Id;
            this.claimNumber = c.Name;
            this.claimAmount = c.Claim_Amount__c;
            this.status = c.Approval_Status__c;

            Policy__c policyRecord = (Policy__c)p;
            if (policyRecord != null && policyRecord.RecordType != null) {
                this.policyType = policyRecord.RecordType.Name;
            } else {
                this.policyType = 'Unknown';
            }

            this.policyHolderName =
                (customerRec != null) ? customerRec.Name : '';

            this.daysOpen = (c.Date_of_Loss__c != null)
                ? (Date.today().daysBetween(c.Date_of_Loss__c.date()))
                : 0;
        }
    }

    @AuraEnabled(cacheable=true)
    public static List<ClaimWrapper> getAssignedClaims() {

        List<ClaimWrapper> results = new List<ClaimWrapper>();

        List<Claim__c> claims = [
            SELECT Id, Name, Claim_Amount__c, Approval_Status__c,
                Date_of_Loss__c, Policy__r.Name,
                Policy__r.RecordType.DeveloperName,
                Policy__r.Customer__r.FirstName,
                Policy__r.Customer__r.LastName
            FROM Claim__c
            WHERE OwnerId = :UserInfo.getUserId()
            ORDER BY Date_of_Loss__c DESC
        ];

        for (Claim__c claim : claims) {
            Policy__c policy = claim.Policy__r;

            Contact customer =
                (policy != null && policy.Customer__r != null)
                ? (Contact)policy.Customer__r
                : new Contact();

            results.add(new ClaimWrapper(claim, policy, customer));
        }

        return results;
    }
}

```

```

@Test
private class ClaimsAdjusterControllerTest {
    @testSetup
    static void setupTestData() {
        // Create a real active test user
        Profile p = [
            SELECT Id
            FROM Profile
            WHERE Name = 'Standard User'
            LIMIT 1
        ];

        User u = new User(
            FirstName = 'Test',
            LastName = 'User',
            Alias = 'tuser',
            Username = 'testuser1234343@example.com.test',
            Email = 'testuser123@example.com',
            TimeZoneSidKey = 'America/Los_Angeles',
            LocaleSidKey = 'en_US',
            EmailEncodingKey = 'UTF-8',
            LanguageLocaleKey = 'en_US',
            ProfileId = p.Id
        );
        insert u;

        Id policyRT =
            Schema.SObjectType.Policy__c.getRecordTypeInfoByDeveloperName()
                .get('Auto')
                .getRecordTypeId();

        Id claimRT =
            Schema.SObjectType.Claim__c.getRecordTypeInfoByDeveloperName()
                .get('Accident')
                .getRecordTypeId();

        Contact con = new Contact(
            FirstName = 'John',
            LastName = 'Doe',
            OwnerId = u.Id
        );
        insert con;

        Policy__c policy = new Policy__c(
            Customer__c = con.Id,
            RecordTypeId = policyRT,
            OwnerId = u.Id,
            VIN__c = '1A903284375893532'
        );
        insert policy;

        Claim__c claim = new Claim__c(
            Policy__c = policy.Id,
            RecordTypeId = claimRT,
            Claim_Amount__c = 5000,
            Date_of_Loss__c = Date.today().addDays(-10),
            OwnerId = u.Id
        );
        insert claim;

        System.debug('Claim inserted: ' + claim.Id);
    }

    @test
    static void testGetAssignedClaims() {
        User u = [
            SELECT Id
            FROM User
            WHERE Alias = 'tuser'
            LIMIT 1
        ];

        System.runAs(u) {
            Test.startTest();

            List<ClaimsAdjusterController.ClaimWrapper> results =
                ClaimsAdjusterController.getAssignedClaims();

            System.debug('Result is: ' + results);
            Test.stopTest();

            ClaimsAdjusterController.ClaimWrapper wrap = results[0];
            System.assertNotEquals(null, wrap.claimId);
        }
    }
}

```

```
System.assertNotEquals(null, wrap.claimNumber);
System.assertEquals('New', wrap.status);
System.assertEquals(5000, wrap.claimAmount);
System.assert(wrap.policyHolderName.contains('John'));
System.assertNotEquals('Unknown', wrap.policyType);
}
}
}
```